



CMSC 180

Introduction to Parallel Computing Second Semester AY 2025-2026

Laboratory Research Problem 01 (LRP01)

Computing the Serial Solution to the Min-Max Transformation of Elements in a Column of a Matrix

Introduction

Given a $m \times n$ matrix $\mathbf{X}$ with m rows and n columns, the Min-Max Transformation (MMT) of $\mathbf{X}$ is a matrix $\mathbf{T}$ such that

$$T_{ij} = (X_{ij} - \min(X_j)) / (\max(X_j) - \min(X_j)) \quad (\text{Equation 1})$$

where $\min(X_j)$ and $\max(X_j)$ are, respectively, the minimum and the maximum element of the j th column of $\mathbf{X}$, for all $i = 1 \dots m$ and $j = 1 \dots n$.

Research Question 1: What do you think is the complexity of solving the MMT of a $n \times n$ square matrix $\mathbf{X}$? (*hint*: CMSC 142)

Research Activity 1: Write a computer program using the programming language of your choice for computing the MMT of an $n \times n$ square matrix $\mathbf{X}$. In other words, transform Equation 1 above into a computer program given $\mathbf{X}$.

How to do it?

1. Write a function `mmt()` that accepts as parameters the matrix $\mathbf{X}$, and transforms $\mathbf{X}$ in place (i.e., you are not to create a temporary matrix to hold intermediate computation). For example, in pseudocode:

```
function mmt(X: matrix, m: integer, n: integer): matrix
begin
    for i:=1 to n do
        for j:=1 to m do
            begin
                X[i,j] := {see equation 1 above};
            end;
        end;
    mmt:=X;
end;
```

2. Write the main program `lab01` that includes the following:
 - (1) Read n as a user input (maybe from a command line or as a data stream);
 - (2) Create a non-zero $n \times n$ square matrix $\mathbf{X}$ whose elements are assigned with random integers (make sure that any integer $i \neq 0$);
 - (3) Take note of the system time `time_before`;
 - (4) transform $\mathbf{X}$ via a call to `mmt(X, n, n)`;
 - (5) Take note of the system time `time_after`;

- (6) Obtain the elapsed time `time_elapsed:=time_after - time_before`;
- (7) output `time_elapsed`;

For example, for computing the MMT of a 100×100 square matrix **X**:

```
$ lab01 < 100
```

```
$ time elapsed: 10.2345 seconds
```

3. Fill in the following table with your time readings:

n	Time Elapsed (seconds)			Average Runtime (seconds)	Complexity*	
	Run 1	Run 2	Run 3		Based on $n = 100$	Based on $\max(n)$
100						
200						
300						
400						
500						
600						
700						
800						
900						
1,000						
2,000						
4,000						
8,000						
16,000						
20,000						

*What does your answer to **Research Question 1** say, but converted into a time with respect to $n = 100$ as well as to $\max(n)$?

Research Question 2: Were you able to run up to $n > 20,000$? If so, can you make it higher to 50,000 or even 100,000? If not, why do you think so and what do you need to do to make it so?

4. Using a graphing software (such as LibreOffice Calc), create a line graph of n versus **Average Runtime** obtained from the Table above. On the same graph, plot n versus **Complexity** as well (both based on $n = 100$ and on $\max(n)$ at least up to the n where your program worked).

Research Question 3: Do the lines agree, at least in the form? If not, provide an explanation why so?

Research Question 4: Discuss ways on how we can make it better (lower average runtime) without using any extra processors or cores (notice that the word “ways” is in plural form).



[CC BY-NC-SA 2026 Jaderick P. Pabico/ICS/UPLB](#)